CASCADED CONVOLUTIONAL NEURAL NETWORK FOR SOLAR PANEL DEFECT CLASSIFICATION

CASE STUDY REPORT
for
21CSC406J – NEURAL NETWORKS AND DEEP LEARNING

Submitted by
Divyanshi Jain [RA2311026010639]

Syed Mohammed Ghouse S [RA2311026010643]

Purvee Singh [RA2311026010645]

In partial fulfilment of the requirements for the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE ENGINEERING
with specialization in AIML



DEPARTMENT OF COMPUTATIONAL INTELLIGENCE
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR- 603 203

NOVEMBER 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR – 603 203

BONAFIDE CERTIFICATE

This is to certify that the Case Study Report titled "**Cascaded Convolutional Neural Network for Solar Panel Defect Classification**" is a bonafide record of work carried out by the following student(s) as part of the course **21CSC406J - NEURAL NETWORKS AND DEEP LEARNING**, during the academic year 2026–2027 Odd Semester.

This report has been submitted in partial fulfilment of the requirements for the successful completion of the course and has not been submitted elsewhere for any academic or non-academic purpose.

The work is found to be satisfactory and is hereby accepted.

Submitted by:

Register Number	Name of Student(s)
RA2311026010639	Divyanshi Jain
RA2311026010643	Syed Mohammed Ghouse S
RA2311026010645	Purvee Singh

Date of Submission: November 2026

Dr. Illakiya T
Supervisor,
Assistant Professor,
Department of Computational Intelligence,
SRMIST, KTR

Dr. R. Annie Uthra,
Professor & Head,
Department of Computational Intelligence,
SRMIST, KTR

ABSTRACT

This case study applies the main ideas of the Neural Networks and Deep Learning course, convolutional neural networks, transfer learning, attention and generative models, to a practical problem: classifying defects of solar photovoltaic modules from 24 x 40 pixel infrared images (12 classes, 20,000 images). A flat classifier struggles with the class imbalance and look-alike faults, so a cascade of five small CNNs is built: a triage network that separates healthy from defective modules, a family network and one specialist per family. The report summarises the theory of the layers, the optimisation and regularisation used, transfer learning with an ImageNet-pretrained EfficientNet-B0 (whose blocks contain squeeze-and-excitation channel attention) and the role GANs could play for rare classes, and maps each concept to the implemented system. On a fixed held-out split the from-scratch cascade reaches 79.8% accuracy, and fine-tuning the pretrained backbone reaches 83.2% accuracy, macro-F1 0.70 and 95.8% on the healthy-versus-defect decision. Removing brightness augmentation, which destroys the temperature signal, was the largest single gain. A flat network with the same backbone is essentially tied with the cascade, and the family decision is the main source of errors. Improvements and optimisation strategies are proposed for the remaining errors.

TABLE OF CONTENTS

CHAPTER NO	TITLE	Page No
1	INTRODUCTION	1
2	THEORETICAL FOUNDATION: CNN, TRANSFER LEARNING, ATTENTION & GANs	3
3	CASE ANALYSIS & IMPLEMENTATION	8
4	PROPOSED IMPROVEMENTS / OPTIMIZATION STRATEGIES	23
5	CONCLUSION AND FUTURE WORK	25
	REFERENCES	26
	APPENDICES	28

LIST OF FIGURES

CHAPTER NO	FIG. NO	Page No
2.1	Architecture of a convolutional neural network: the Stage 1 triage CNN of the case study	3
2.2	Growth of the receptive field through the layers of the triage CNN	4
2.3	Transfer learning from ImageNet to the thermal defect task	5
2.4	Squeeze-and-excitation channel attention, as used inside EfficientNet-B0	6
2.5	Generative adversarial network: generator and discriminator	6
3.1	Class distribution of the dataset (logarithmic axis)	11
3.2	One test image per class (inferno colour map)	11
3.3	The implemented three-stage cascade	12
3.4	Test accuracy and macro-F1 by configuration	17
3.5	Validation macro-F1 and training loss of the five stage networks	18
3.6	Where the cascade makes its errors	20
3.7	Per-class F1 on the test set	20
3.8	Confusion matrix of the pretrained cascade (row-normalised, counts inside cells)	21

A.1	Best validation macro-F1 of each stage network, from scratch and pretrained	32
A.2	Output of the live demonstration for random test images (green: correct, red: wrong)	33

LIST OF TABLES

CHAPTER NO	TABLE. NO	Page No
2.1	The four concept families and their role in the case study	7
3.1	Dataset description and stratified split	8
3.2	Architecture summary of the stage networks	12
3.3	Training configuration	13
3.4	Mapping of course concepts to the case study	14
3.5	Test results by configuration	15
3.6	Test errors per stage (out of 3,000 images)	19
3.7	Most frequent confusions of the pretrained cascade	21
4.1	Proposed improvements and optimisation strategies	24
A.1	Layer-by-layer summary of CNN-1 (from-scratch variant, generated from the implemented model)	28

CHAPTER 1
INTRODUCTION

1.1 Background

Photovoltaic (PV) power plants contain thousands of modules, and defects such as hot spots, cracks, soiling, shadowing and bypass-diode failures lower their output and can cause fires. Manual thermal inspection does not scale, so drones with infrared (IR) cameras are used to image the modules, and a classifier has to label each module automatically. In an IR image brightness encodes temperature: a healthy module is heated almost uniformly, while a defect creates a localised or characteristic thermal pattern (Millendorf et al., 2020).

Deep learning is well suited to this recognition task, and the case touches the main topics of the course. Convolutional neural networks (CNNs) extract spatial features, transfer learning reuses features learned on ImageNet, attention mechanisms re-weight features inside modern backbones, and generative adversarial networks (GANs) offer a way to create synthetic examples of rare classes. This report studies how these ideas behave on one concrete dataset, the InfraredSolarModules benchmark.

1.2 Problem Statement

Given a single-channel IR image of 24 x 40 pixels showing one PV module, classify it into one of 12 classes: No-Anomaly and eleven fault types. Three properties make the problem hard: half of the images are healthy while the rarest faults have fewer than 250 images; several faults differ only in how many cells or diodes are affected; and a drone needs a model that is small and fast. A single network that chooses among all 12 classes (a flat classifier) therefore tends to plateau and to be biased towards the healthy class.

1.3 Objectives

- **O1.** To explain the neural-network concepts relevant to the case: CNN layers, optimisation and regularisation, transfer learning, attention and GANs.
- **O2.** To design and train a cascade of five small CNNs (healthy versus defect, fault family, fault subtype) and a pretrained variant, and to compare it with a flat classifier.
- **O3.** To map every concept to the implemented system, and to measure the effect of the main design choices with controlled experiments.

- **O4.** To analyse where the model fails and to propose improvement and optimisation strategies.

The report follows this order: Chapter 2 gives the theory, Chapter 3 describes and analyses the case, Chapter 4 proposes improvements, and Chapter 5 concludes.

CHAPTER 2
THEORETICAL FOUNDATION: CNN, TRANSFER LEARNING, ATTENTION & GANs

2.1 Convolutional Neural Networks

A CNN processes an image with layers that share weights across positions, which exploits two properties of images: nearby pixels are correlated and the same pattern can occur anywhere (Goodfellow et al., 2016). The stage networks of the case have the structure shown in Figure 2.1.

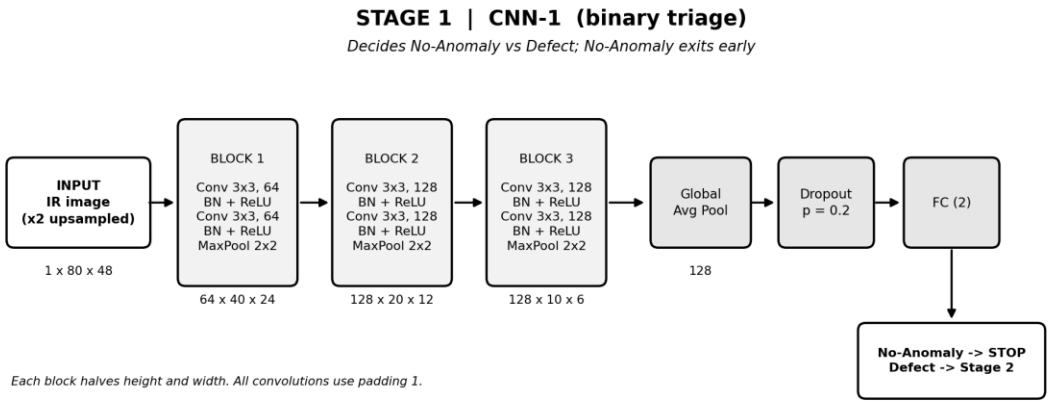


Fig. 2.1. Architecture of a convolutional neural network: the Stage 1 triage CNN of the case study.

2.1.1 Convolution, Normalisation, Activation and Pooling

A convolutional layer with C_in input and C_out output channels and a k x k kernel computes, for every output channel c and position (i, j),

y[c, i, j] = b[c] + sum over (m, u, v) of W[c, m, u, v] x x[m, i + u, j + v]

with k^2 x C_in x C_out weights. Batch normalisation (Ioffe & Szegedy, 2015) standardises each channel over the mini-batch and rescales it with learned parameters, y = gamma x (x - mean) / sqrt(var + eps) + beta, which stabilises training. The ReLU activation f(x) = max(0, x) adds non-linearity without saturating, and 2 x 2 max pooling halves the spatial size and gives tolerance to small shifts. Global average pooling replaces flattening by the mean of each feature map, z_c = mean of x_c(i, j), which keeps the classifier small, and a fully connected layer with softmax gives class probabilities p_k = exp(s_k) / sum_j exp(s_j).

2.1.2 Receptive Field and Feature Hierarchy

Stacked small convolutions and pooling enlarge the region of the input that a unit can see. Each layer adds (k - 1) x jump to the receptive field, and a stride s multiplies the jump by s. For the three blocks of the triage network the receptive field grows from 3 to 36 pixels (Figure 2.2), so that early layers respond to edges and intensity gradients of a heated region, middle layers to blobs and their shape, and the last layer to the arrangement of regions across the module.

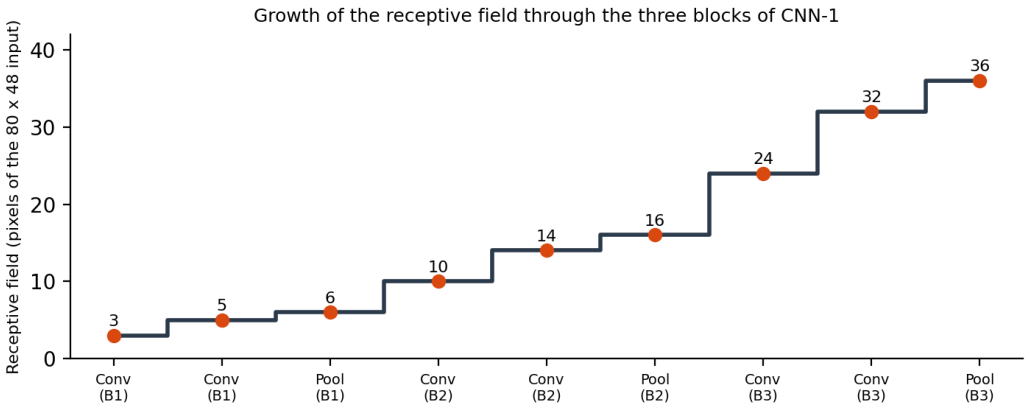


Fig. 2.2. Growth of the receptive field through the layers of the triage CNN.

2.1.3 Training: Loss, Backpropagation and Adam

The networks minimise the class-weighted cross-entropy L = -(1/B) sum_i w[y_i] log p[i, y_i], with weights w_r = N / (K x n_r) that give rare classes a larger gradient. Backpropagation applies the chain rule layer by layer; for softmax with cross-entropy the gradient with respect to the scores is dL/ds_k = w_y (p_k - 1[k = y]). Weights are updated with Adam (Kingma & Ba, 2015), which keeps running averages of the gradient and of its square:

$$m_t = 0.9\, m_{(t-1)} + 0.1\, g_t, \quad v_t = 0.999\, v_{(t-1)} + 0.001\, g_t^2, \quad \theta \leftarrow \theta - \alpha \times \hat{m} / (\sqrt{\hat{v}} + \varepsilon)$$

so that each weight gets its own step size. The learning rate is halved when the validation score stalls.

2.1.4 Regularisation and Initialisation

Dropout zeroes each unit with probability p during training and rescales the others (Srivastava et al., 2014); weight decay adds an L2 penalty; early stopping keeps the checkpoint with the best validation score; flips enlarge the training set. Weights of the from-scratch convolutions use the Kaiming scheme, with variance 2 / fan_out, which keeps the activation variance stable through ReLU layers (He

et al., 2015). Together these control the bias-variance trade-off: a small network is biased, a large one fits noise.

2.2 Transfer Learning

Transfer learning initialises a network with weights learned on a large source task. Early convolutional layers learn generic features (edges, gradients, textures) that transfer across tasks, while later layers are more specific to the source classes (Yosinski et al., 2014). Fine-tuning keeps the feature extractor, replaces the classifier head and trains all layers with a small learning rate (Figure 2.3). This reduces the number of target samples needed and gives the optimiser a good starting region.

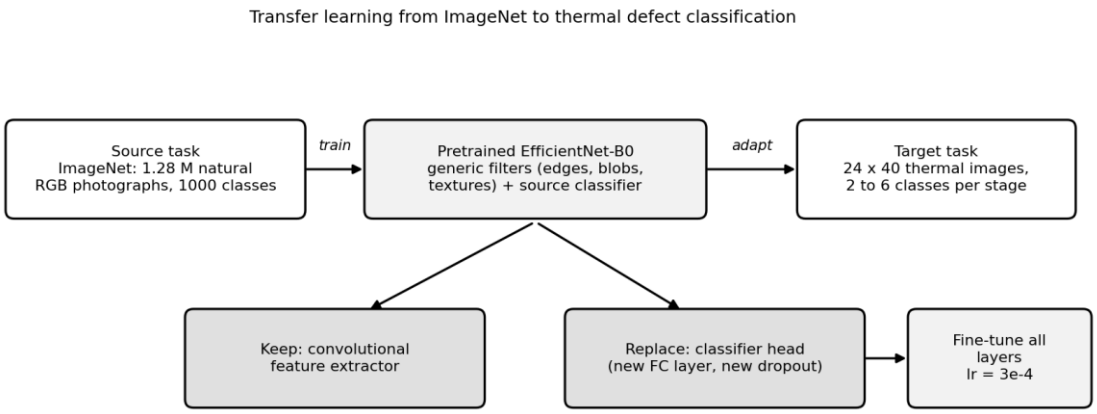


Fig. 2.3. Transfer learning from ImageNet to the thermal defect task.

The pretrained backbone used here is EfficientNet-B0 (Tan & Le, 2019), built from mobile inverted-bottleneck (MBConv) blocks with depthwise convolutions and residual connections (cf. He et al., 2016), and scaled by a compound coefficient of depth, width and resolution. It has about 4 million parameters. The source images (ImageNet; Deng et al., 2009) are natural RGB photographs, so a domain shift exists: the grayscale thermal channel is repeated three times and the image resized to 160 x 96 to match the scale the filters were trained for.

2.3 Attention Mechanisms

Attention lets a network weight parts of its representation by their relevance. Channel attention, as in squeeze-and-excitation (SE) blocks (Hu et al., 2018), pools each channel to a number, passes the vector through two fully connected

layers and a sigmoid to obtain a weight per channel, and multiplies the feature map by these weights (Figure 2.4):

$$s = sigmoid(W2 \; x \; ReLU(W1 \; x \; z)), \quad x' = s \; (channel\text{-}wise) \; x$$

Squeeze-and-excitation: channel attention inside the MBConv blocks of EfficientNet-B0

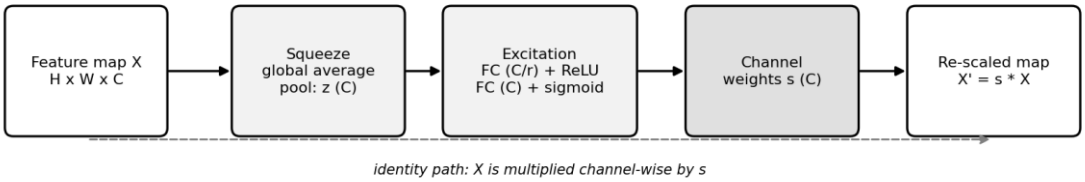


Fig. 2.4. Squeeze-and-excitation channel attention, as used inside EfficientNet-B0.

Self-attention, the core of Transformers (Vaswani et al., 2017) and Vision Transformers (Dosovitskiy et al., 2021), computes softmax(Q K^T / sqrt(d)) V, so that every position can attend to every other. In this case study attention is used implicitly: every MBConv block of the pretrained backbone contains an SE module. Self-attention is a candidate for improving the family decision (Chapter 4).

2.4 Generative Adversarial Networks

A GAN (Goodfellow et al., 2014) trains a generator G, which maps noise z to an image, against a discriminator D, which decides whether an image is real or generated, in a minimax game:

$$\min_G \max_D \; E[\log D(x)] + E[\log(1 - D(G(z)))]$$

Generative adversarial network: G and D play a minimax game

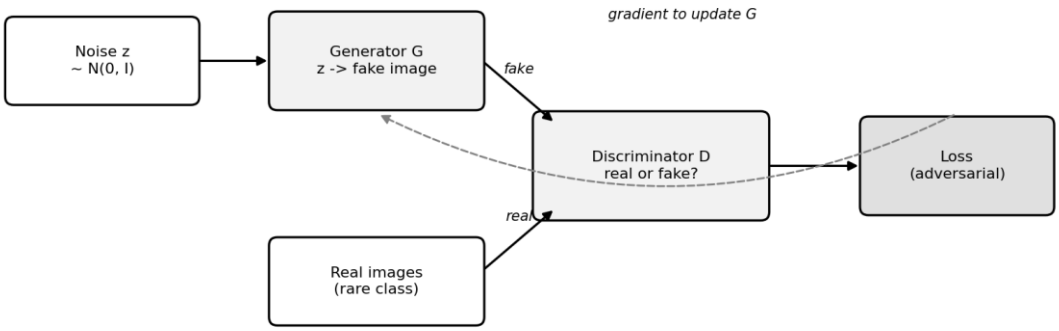


Fig. 2.5. Generative adversarial network: generator and discriminator.

When training converges the generator produces images that follow the data distribution, which makes GANs a tool for synthetic augmentation of rare classes. GANs are not used in the implemented system, because the rare classes have only 123 to 174 training images, too few to train a stable GAN; they are proposed as an improvement in Chapter 4.

2.5 Summary of the Concepts

Table 2.1: The four concept families and their role in the case study

Concept	Key idea	Role in this case study
CNN	Weight-sharing convolutions, pooling, hierarchical features	All five stage networks; from-scratch variant
Transfer learning	Reuse features from a source task and fine-tune	Pretrained EfficientNet-B0 at every stage (+3.4 points)
Attention	Re-weight features by relevance (channel or self-attention)	SE channel attention inside EfficientNet-B0; self-attention proposed for Stage 2
GAN	Generator and discriminator in a minimax game	Not used; proposed for synthetic rare-class images

CHAPTER 3
CASE ANALYSIS & IMPLEMENTATION

3.1 Case Description and Dataset

The InfraredSolarModules dataset (Millendorf et al., 2020) contains 20,000 grayscale IR images of 24 pixels wide and 40 tall, captured on commercial drone flights, with 12 classes (Table 3.1). Half of the images are healthy, and the four rarest classes have 175 to 249 images (Figure 3.1). Figure 3.2 shows one test image per class. The images are split once, stratified by class with a fixed seed, into 70% training, 15% validation and 15% test images, and the test images are never used for training, early stopping, normalisation or tuning.

Table 3.1: Dataset description and stratified split

Class	**Family**				
No-Anomaly	Healthy				
Cell	Electrical				
Cell-Multi	Electrical				
Diode	Electrical				

Class	Family				
Diode-Multi	Electrical				
Hot-Spot	Electrical				
Hot-Spot-Multi	Electrical				
Soiling	Environmental				
Shadowing	Environmental				

Class	**Family**				
Vegetation	Environmental				
Cracking	Structural				
Offline-Module	Structural				
Total					

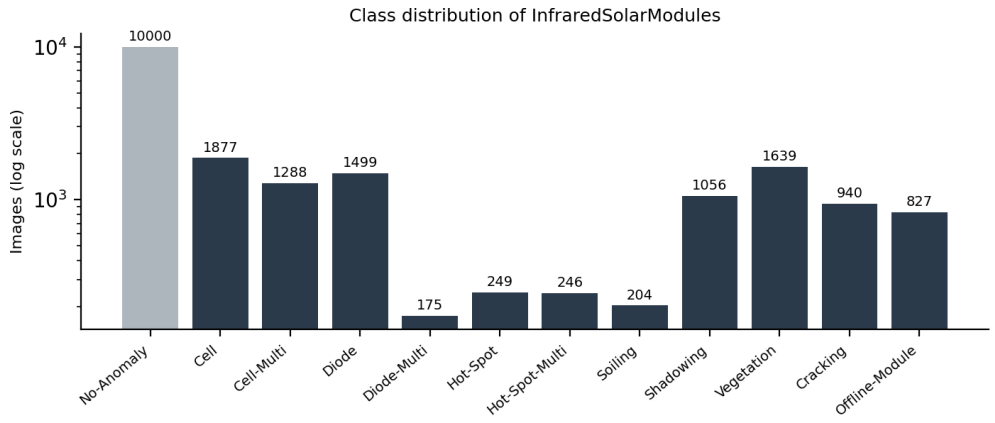


Fig. 3.1. Class distribution of the dataset (logarithmic axis).

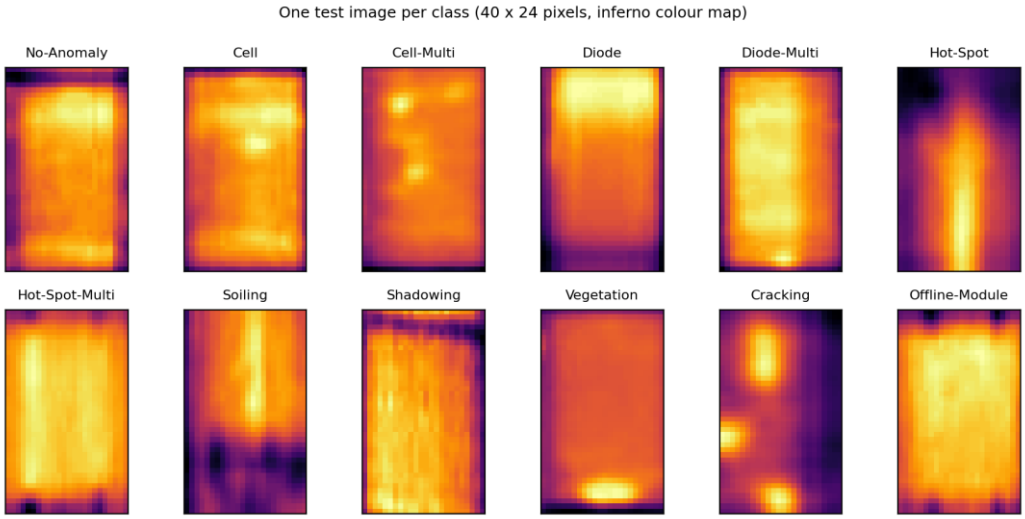


Fig. 3.2. One test image per class (inferno colour map).

3.2 Model Architecture and Training

3.2.1 The Cascade

Instead of one 12-way classifier the system uses five small networks in three stages (Figure 3.3). CNN-1 decides healthy or defective and lets healthy modules exit early. CNN-2 assigns a defect to one of three families (Electrical, Environmental, Structural). One specialist per family (CNN-3a, 3b, 3c with 6, 3 and 2 classes) names the exact fault. Each network is trained independently on the data matching its task, and the cascade is assembled at test time, so a stage's error is measured as it would occur in use.

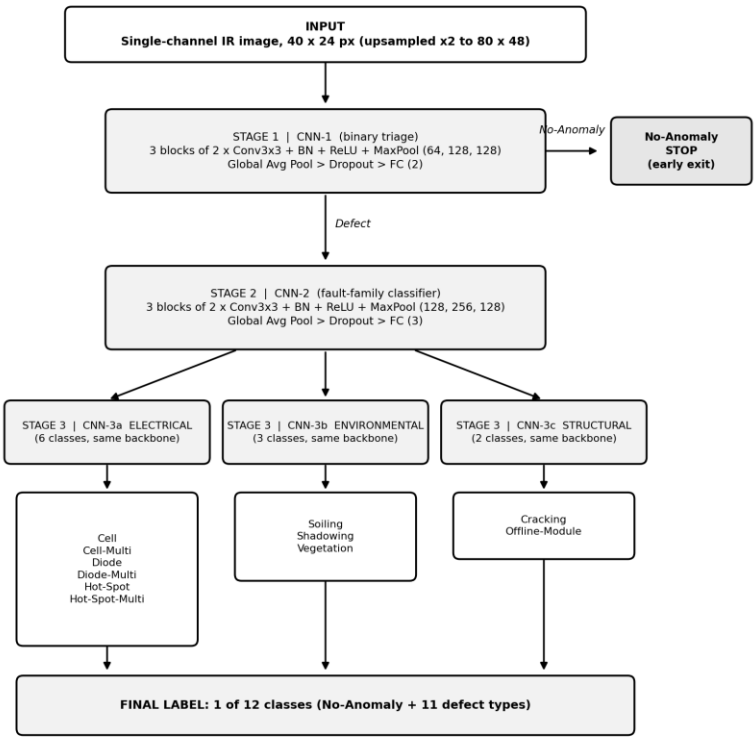


Fig. 3.3. The implemented three-stage cascade.

3.2.2 Network Structure

In the from-scratch variant every network has three blocks of two 3 x 3 convolutions (BatchNorm and ReLU after each) followed by 2 x 2 max pooling, then global average pooling, dropout and a fully connected layer (Table 3.2). The input is upsampled to 80 x 48 so that small defects survive three pooling steps. In the pretrained variant every network is an ImageNet-pretrained EfficientNet-B0 with a new head.

Table 3.2: Architecture summary of the stage networks

	Task	**Channels, dropout**		**Parameters (scratch / pretrained)**
	Healthy vs defect	64-128-128, 0.2		555,074 / 4,010,110
	Family	128-256-		1,478,147 / 4,011,391

	Task	Channels, dropout		Parameters (scratch / pretrained)
		128, 0.4		
	Electrical subtype	128-256-128, 0.4		1,478,534 / 4,015,234
	Environmental subtype	128-256-128, 0.4		1,478,147 / 4,011,391
	Structural subtype	128-256-128, 0.4		1,478,018 / 4,010,110

3.2.3 Training

Training uses Adam (weight decay 1e-4, batch size 64), a class-weighted cross-entropy loss, random flips as the only augmentation, and early stopping on validation macro-F1. Table 3.3 lists the settings.

Table 3.3: Training configuration

Setting	From scratch	Pretrained
Input size	80 x 48 (2x upsampling)	160 x 96, channel repeated 3 times
Initialisation	Kaiming normal (fan-out)	ImageNet weights
Learning rate	5e-4	3e-4
Early stopping patience (maximum epochs)	25 (150)	6 (25)

Setting	From scratch	Pretrained
Precision	32-bit	Mixed precision
Loss	Class-weighted cross-entropy	Class-weighted cross-entropy
Hardware	NVIDIA RTX 5060 Laptop GPU	NVIDIA RTX 5060 Laptop GPU

3.3 Mapping Concepts to the Case

Table 3.4 maps each concept of Chapter 2 to the place where it appears in the implementation and to what was observed.

Table 3.4: Mapping of course concepts to the case study

Concept	Where it appears	Observation
Convolution, BatchNorm, ReLU, pooling	Every block of the stage networks	Three blocks give a receptive field of 36 pixels (Fig. 2.2)
Class-weighted cross-entropy	Loss of all networks	Rare classes are not ignored, but stay the weakest
Adam and learning-rate schedule	Optimiser of all networks	Pretrained networks converge in 10 to 20 epochs
Dropout, weight decay, early stopping	All networks	From-scratch family network has noisy validation curves (high variance)
Data augmentation	Random flips	Brightness and noise jitter lowered accuracy by about 10 points
Transfer learning	EfficientNet-B0 at every stage	79.8% to 83.2% accuracy
Attention	SE modules inside EfficientNet-B0	Used implicitly; not isolated in an ablation

Concept	Where it appears	Observation
GAN	Not implemented	Proposed for rare classes (Chapter 4)

3.4 Results and Discussion

All numbers are measured on the 3,000 test images. The best result is the pretrained cascade with 83.2% accuracy, macro-F1 0.700, Cohen's kappa 0.764 and 95.8% on the healthy-versus-defect decision (Table 3.5, Figure 3.4). Figure 3.5 shows the training behaviour of the five networks.

Table 3.5: Test results by configuration

Configuration					
C1 Default (jitter on)					
C2 No photometric jitter					

Configuration					
C3 Improved (80x48, deeper)					
C4 Improved2 (balanced sampling)					
C5 Improved3 (bigger Stage 1/2)					
C6 Data-driven families					
C7 Pretrained EfficientNet-B0					

Configuration					
Raw-pixel Random Forest					
EfficientNet-B0 features + SVM					

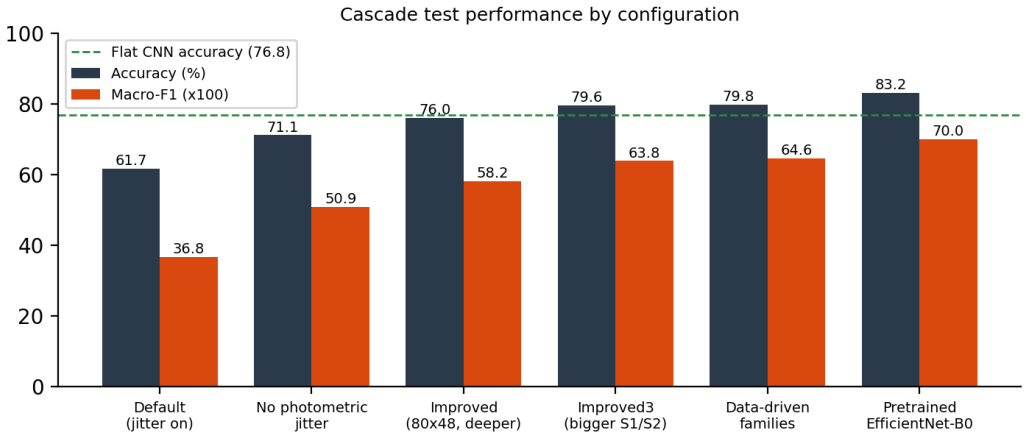


Fig. 3.4. Test accuracy and macro-F1 by configuration.

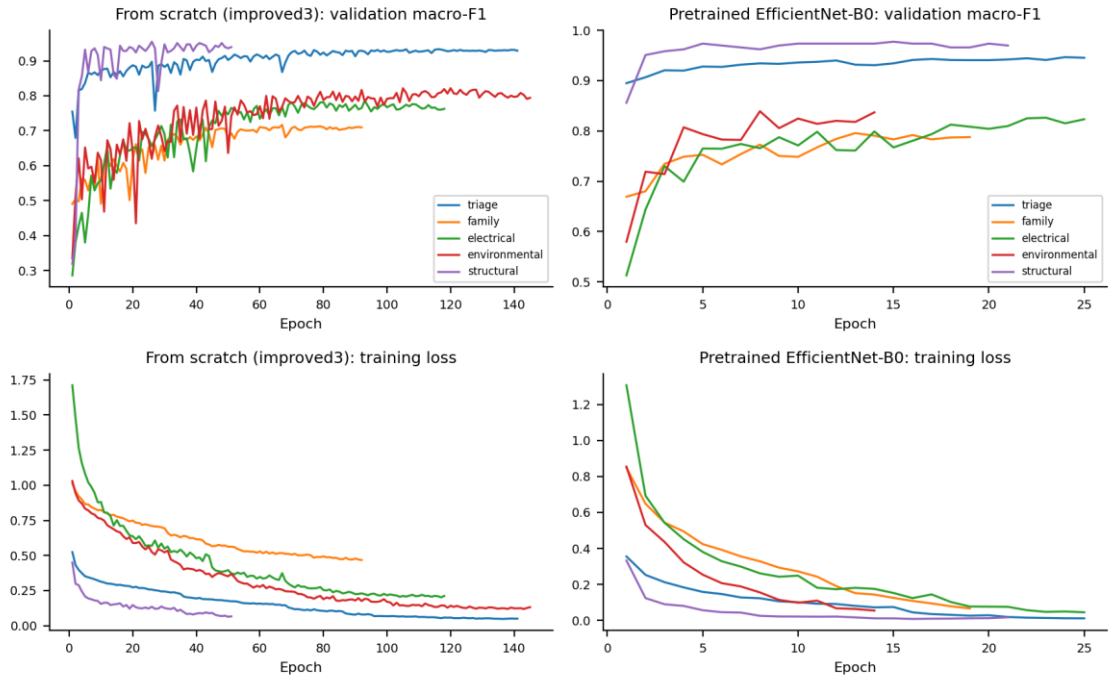


Fig. 3.5. Validation macro-F1 and training loss of the five stage networks.

- **Augmentation.** Removing brightness, contrast and noise jitter raised accuracy from 61.7% to 71.1%. In thermal images brightness is temperature, so the jitter violates the assumption that augmentations preserve the label.
- **Capacity and input size.** Upsampling and two convolutions per block gave 76.0%, and larger front stages 79.6%: a reduction of bias.
- **Transfer learning.** Pretraining raised the cascade to 83.2% and the flat network from 76.8% to 82.6%, so that the two designs end up tied. The pretrained networks converge faster and more smoothly.
- **Cascade versus flat.** 83.2% / 0.700 (cascade) against 82.6% / 0.708 (flat) in accuracy and macro-F1, with five times the parameters for the cascade. The cascade did not show a clear advantage.
- **Did not help.** Soft routing, test-time augmentation, threshold tuning and balanced sampling.

Comparison with the literature is limited by different protocols. Duranay (2023) reports 93.93% accuracy for 12 classes with 10-fold cross-validation and an ImageNet-pretrained EfficientNet-B0 with a feature selection that uses the labels; the same kind of pipeline run on our held-out split scored 78.7%, below our cascade. For the binary decision, Alves et al. (2021) report 92.5% with cross-validation, against 95.8% for our Stage 1. A random forest on our split scores

61.6%, against 84.0% reported by Corley et al. (2025), whose protocol is not stated. We therefore claim parity for the binary task and do not claim to beat the published 12-class figures.

3.5 Error Analysis

3.5.1 Errors by Stage

Every error of the cascade can be attributed to the first stage that went wrong (Table 3.6, Figure 3.6). Stage 2, the family decision, causes more than half of all errors, and an error there cannot be corrected by Stage 3.

Table 3.6: Test errors per stage (out of 3,000 images)

Configuration	**[illegible]**	**Stage 2 wrong family**	**Stage 3 wrong subtype**	
From scratch	[illegible]	300	126	
Pretrained	**[illegible]**	**272**	**106**	

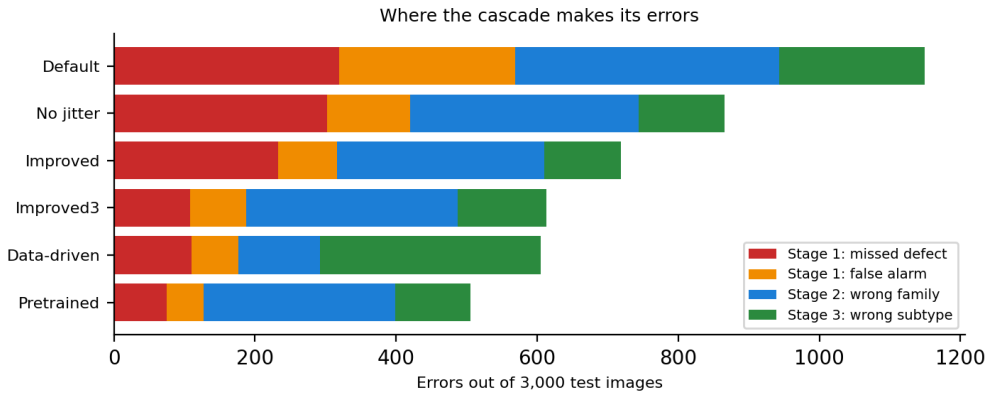


Fig. 3.6. Where the cascade makes its errors.

3.5.2 Per-Class Behaviour and Confusion

Rare classes are the weakest (Figure 3.7). The confusion matrix of the pretrained cascade (Figure 3.8) shows that errors concentrate between classes of one physical phenomenon, a single versus several affected cells or diodes, and between faint hot spots and the healthy class.

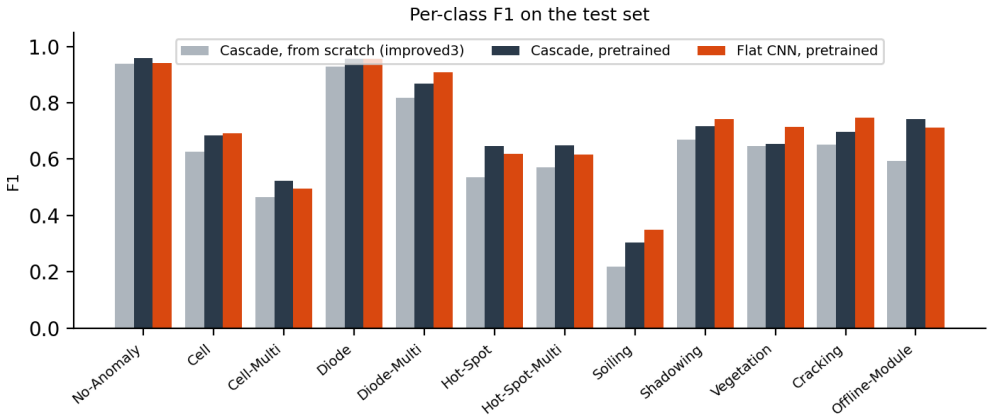


Fig. 3.7. Per-class F1 on the test set.

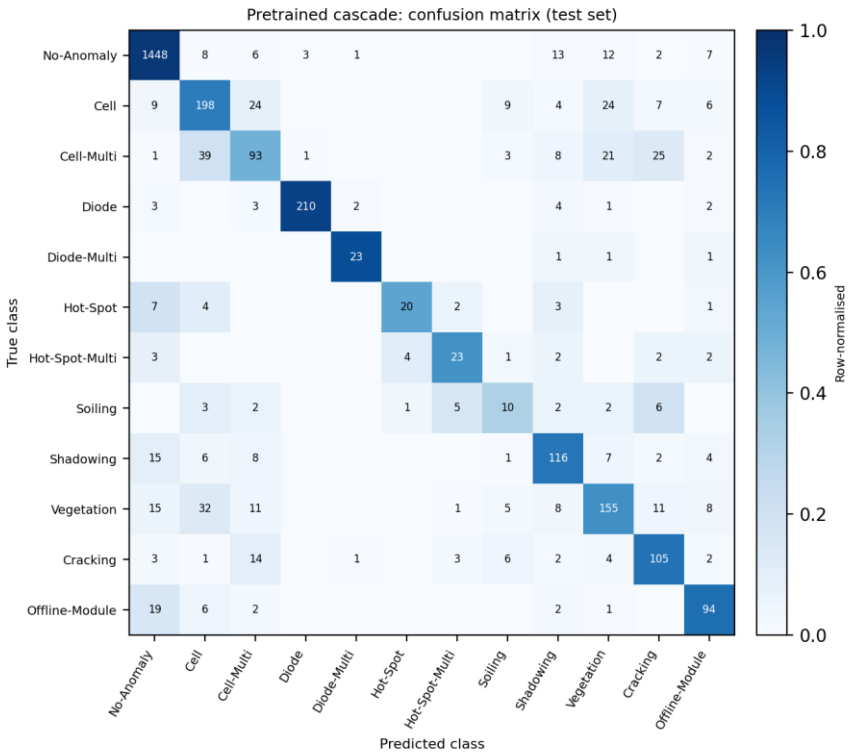


Fig. 3.8. Confusion matrix of the pretrained cascade (row-normalised, counts inside cells).

Table 3.7: Most frequent confusions of the pretrained cascade

True class	Predicted as		Share of the true class
Cell-Multi	Cell		20%
Vegetation	Cell		13%
Cell-Multi	Cracking		13%
Cell	Vegetation		9%
Cell	Cell-Multi		9%
Cell-Multi	Vegetation		11%

3.5.3 Interpretation

The errors are explained by the concepts of Chapter 2. High variance of the small family and electrical networks on few training images is reduced by pretraining but not removed. Error propagation between independently trained stages means that a misrouted image meets a specialist that has never seen its class. And with only 123 to 174 training images, rare classes do not constrain the weights enough, whatever the loss weights.

CHAPTER 4
PROPOSED IMPROVEMENTS / OPTIMIZATION STRATEGIES

The analysis points to three weaknesses: the family decision, the rare classes and the cost of the cascade. The following strategies are proposed. They are not implemented in the case study, and the expected effects are hypotheses to be tested.

4.1 Better Family Decision

Train CNN-2 jointly with the specialists, so that the stages are optimised for the final label, or train the specialists on the routing errors of CNN-2. Self-attention over the pooled feature map, in the style of a small Vision Transformer (Dosovitskiy et al., 2021), could help to relate the positions of heated regions across the module, which matters for the family.

4.2 Rare Classes: GAN and Class-Specific Augmentation

Train a conditional GAN or a lightweight variational model on the rare classes (Soiling, Hot-Spot, Diode-Multi) to create synthetic images, or use class-specific geometric augmentation and mix-up. The generator would be evaluated on a held-out validation set, and its images used only for training. With 123 to 174 real images, a pretrained or small generator and strong regularisation of the discriminator would be needed.

4.3 Loss and Optimisation

Replace the class weights by focal loss, which down-weights easy examples, or by label-distribution-aware margins. Tune the learning rate, weight decay and dropout with a search on the validation split, and use a cosine learning-rate schedule with warm-up for the fine-tuned networks.

4.4 Model Compression and Efficiency

The pretrained cascade has five EfficientNet-B0 networks (20 million parameters). Knowledge distillation into the small from-scratch networks, quantisation to 8 bits and a shared backbone with five heads would reduce memory and latency so that the early exit pays off on a drone.

4.5 Evaluation Strategy

Run several seeds and a leak-free 10-fold cross-validation, deduplicate near-identical crops before splitting, and report confidence intervals. Ensembles of the cascade and the flat network could combine their complementary errors.

Table 4.1: Proposed improvements and optimisation strategies

Strategy	Problem addressed	Expected effect	Cost
Joint or error-aware training of the stages	Stage 2 errors (272 of 505)	Fewer wrong-family errors	Longer training, more tuning
Self-attention for the family decision	Look-alike families	Better use of the global layout	More parameters
GAN or class-specific augmentation	Rare classes	Higher macro-F1	Unstable GAN training on 123 to 174 images
Focal loss, tuned schedule	Easy-example dominance, tuning noise	Small accuracy gain	Low
Distillation, quantisation, shared backbone	Cost of five networks	Lower latency and memory	Possible small accuracy loss
Cross-validation, deduplication, seeds	Comparability with the literature	Reliable estimates	10x training time

CHAPTER 5
CONCLUSION AND FUTURE WORK

5.1 Conclusion

The case study applied CNNs, transfer learning and attention-based backbones to classify solar module defects from tiny infrared images. A cascade of five small CNNs reached 79.8% accuracy from scratch and 83.2% with an ImageNet-pretrained EfficientNet-B0, with 95.8% on the healthy-versus-defect decision. Three lessons follow from the experiments.

- **1.** Augmentation must preserve the label-relevant signal: brightness is temperature in thermal images, and jittering it cost about ten points.
- **2.** Transfer learning helps even on 24 x 40 pixel thermal images: it raised accuracy by 3.4 points and made training fast and smooth.
- **3.** A cascade is not automatically better than a flat network. With the same backbone the two tie, the family decision causes more than half of the errors, and the cascade has five times the parameters.

Limitations: single runs with one seed, a cross-validation that was prepared but not completed, weak rare classes, near-duplicate images in the dataset, and no measured efficiency gain. The 93% ceiling and the 95% target set at the start were not reached under the strict held-out protocol.

5.2 Future Work

Implement and test the strategies of Chapter 4, in particular joint training of the stages, class-specific augmentation or a GAN for the rare classes, and distillation for edge deployment. Evaluate with several seeds and a leak-free cross-validation, measure latency on drone hardware, and investigate additional information such as RGB images or neighbouring panels.

REFERENCES

Alves, R. H. F., de Deus Junior, G. A., Marra, E. G., & Lemos, R. P. (2021). Automatic fault classification in photovoltaic modules using convolutional neural networks. Renewable Energy, 179, 502-516.

Corley, et al. (2025). Aerial infrared health monitoring of solar photovoltaic farms at scale. CVPR Workshops (PBVS). arXiv:2503.02128.

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

Dosovitskiy, A., et al. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. International Conference on Learning Representations (ICLR).

Duranay, Z. B. (2023). Fault detection in solar energy systems: A deep learning approach. Electronics, 12(21), 4397. https://doi.org/10.3390/electronics12214397

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press.

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). Generative adversarial nets. Advances in Neural Information Processing Systems, 27.

He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. Proceedings of the IEEE International Conference on Computer Vision (ICCV).

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 770-778.

Hu, J., Shen, L., & Sun, G. (2018). Squeeze-and-excitation networks. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. Proceedings of the International Conference on Machine Learning (ICML).

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. International Conference on Learning Representations (ICLR).

Millendorf, M., Obropta, E., & Vadhavkar, N. (2020). Infrared solar module dataset for anomaly detection. ICLR 2020 Workshop on AI for Earth Sciences.

Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems, 32.

Pathak, S. P., & Patil, S. A. (2025). Enhanced fault identification in solar panels through binary cascaded convolutional classifiers with thermal-visual augmentation. International Journal of Image, Graphics and Signal Processing, 17(4).

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. Journal of Machine Learning Research, 15, 1929-1958.

Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. Proceedings of the International Conference on Machine Learning (ICML). arXiv:1905.11946.

Tella, H., Hussein, A., Rehman, S., et al. (2025). PV panel defect classification using deep learning ensemble methods. Case Studies in Thermal Engineering, 66.

Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30.

Yosinski, J., Clune, J., Bengio, Y., & Lipson, H. (2014). How transferable are features in deep neural networks? Advances in Neural Information Processing Systems, 27.

APPENDICES

A. Source code and repository

The complete source code, configuration files, trained Stage 3 checkpoints, raw results, tests and this report are available in the project repository: https://github.com/techwack/cascaded-cnn. The main entry points are python -m solarnet.train --node all --config configs/pretrained.yaml for training, python -m solarnet.evaluate --flat --config configs/pretrained.yaml for evaluation and python scripts/predict.py --sample 5 for the live demonstration.

B. Model summary output

Table A.1: Layer-by-layer summary of CNN-1 (from-scratch variant, generated from the implemented model)

Layer	Output shape	Parameters
Input	1 x 80 x 48	0
Conv2d(64)	64 x 80 x 48	576
BatchNorm2d	64 x 80 x 48	128
ReLU	64 x 80 x 48	0
Conv2d(64)	64 x 80 x 48	36,864
BatchNorm2d	64 x 80 x 48	128
ReLU	64 x 80 x 48	0

Layer	Output shape	Parameters
MaxPool2d	64 x 40 x 24	0
Conv2d(128)	128 x 40 x 24	73,728
BatchNorm2d	128 x 40 x 24	256
ReLU	128 x 40 x 24	0
Conv2d(128)	128 x 40 x 24	147,456
BatchNorm2d	128 x 40 x 24	256
ReLU	128 x 40 x 24	0
MaxPool2d	128 x 20 x 12	0
Conv2d(128)	128 x 20 x 12	147,456
BatchNorm2d	128 x 20 x 12	256
ReLU	128 x 20 x 12	0

Layer	Output shape	Parameters
Conv2d(128)	128 x 20 x 12	147,456
BatchNorm2d	128 x 20 x 12	256
ReLU	128 x 20 x 12	0
MaxPool2d	128 x 10 x 6	0
GlobalAvgPool	128	0
FC(2)	2	258
Total		**555,074**

C. Key source code

Listing A.1: Cascade inference with early exit (src/solarnet/models.py)

```
    def predict(self, x: torch.Tensor, threshold: float = 0.5, tta: bool = False) ->
dict[str, torch.Tensor]:
        """Route a batch through the cascade (hard routing, with early exit).


        `threshold` is the triage P(defect) above which an image continues to stage 2.
        Returns global class predictions plus the per-stage decisions, so
        evaluation can attribute errors to the stage that made them.
        """
        n = x.shape[0]
        pred = torch.full((n,), self.healthy_index, dtype=torch.long, device=x.device)
        fam_pred = torch.full((n,), -1, dtype=torch.long, device=x.device)


        is_defect = self._probs(self.triage, x, tta)[:, 1] > threshold
```

```
        idx = is_defect.nonzero(as_tuple=True)[0]
        if idx.numel():  # early exit: healthy panels never reach stage 2
            fam_pred[idx] = self._probs(self.family, x[idx], tta).argmax(1)
            for f_i, fam in enumerate(self.family_names):
                sub = idx[fam_pred[idx] == f_i]
                if sub.numel():
                    pred[sub] = self.class_offsets[fam] + self._probs(self.fine[fam],
x[sub], tta).argmax(1)
        return {"pred": pred, "is_defect": is_defect, "family": fam_pred}

    @torch.no_grad()
```

Listing A.2: Class weights for the imbalanced loss (src/solarnet/data.py)

```
def class_weights(targets: np.ndarray, num_classes: int) -> np.ndarray:
    """w_r = N / (K * n_r); classes absent from `targets` get weight 0."""
    counts = np.bincount(targets, minlength=num_classes).astype(np.float64)
    with np.errstate(divide="ignore"):
        w = np.where(counts > 0, len(targets) / (num_classes * counts), 0.0)
    return w.astype(np.float32)
```

Listing A.3: Attribution of errors to stages (src/solarnet/evaluate.py)

```
def stage_errors(y: np.ndarray, out: dict, tax: Taxonomy) -> dict:
    """Attribute every cascade mistake to the first stage that went wrong."""
    healthy = tax.index(tax.healthy)
    true_defect = y != healthy
    true_fam = np.array([tax.family_names.index(tax.family_of(tax.classes[c])) if c !=
healthy else -1 for c in y])
    is_defect = out["is_defect"].numpy()
    fam = out["family"].numpy()
    wrong = out["pred"].numpy() != y

    triage_miss = true_defect & ~is_defect           # defect exited early as healthy
    triage_false_alarm = ~true_defect & is_defect    # healthy sent down the cascade
    family_wrong = true_defect & is_defect & (fam != true_fam)
    fine_wrong = wrong & ~triage_miss & ~triage_false_alarm & ~family_wrong
    return {
        "stage1_missed_defects": int(triage_miss.sum()),
        "stage1_false_alarms": int(triage_false_alarm.sum()),
        "stage2_wrong_family": int(family_wrong.sum()),
        "stage3_wrong_subtype": int(fine_wrong.sum()),
        "total_errors": int(wrong.sum()),
        "early_exit_rate": float((~is_defect).mean()),
    }
```

D. Hyper-parameter configuration

Listing A.4: configs/pretrained.yaml

```
# Pretrained cascade: an ImageNet-pretrained EfficientNet-B0 under every stage, fine-
tuned.
# Inputs are resized inside the model (to `size`), so data.upsample stays 1.
data:
  root: data/InfraredSolarModules     # folder containing module_metadata.json
  splits_file: data/splits.json       # written by scripts/make_splits.py
  seed: 42
  val_frac: 0.15
  test_frac: 0.15
  upsample: 1

augment:
  hflip: 0.5
  vflip: 0.5
  brightness: 0.0
  contrast: 0.0
  noise_std: 0.0
```

```
train:
  batch_size: 64
  epochs: 25
  lr: 3.0e-4
  weight_decay: 1.0e-4
  patience: 6                          # early stopping on val macro-F1
  weighted_loss: true                  # w_r = N / (K * n_r)
  num_workers: 2
  amp: true                            # mixed precision, ~2x faster on GPU
  out_dir: runs_pretrained

models:
  triage:     {pretrained: true, size: [160, 96], dropout: 0.2}
  family:     {pretrained: true, size: [160, 96], dropout: 0.3}
  fine:       {pretrained: true, size: [160, 96], dropout: 0.3}
  flat:       {pretrained: true, size: [160, 96], dropout: 0.3}
```

E. Additional training results and demonstration output

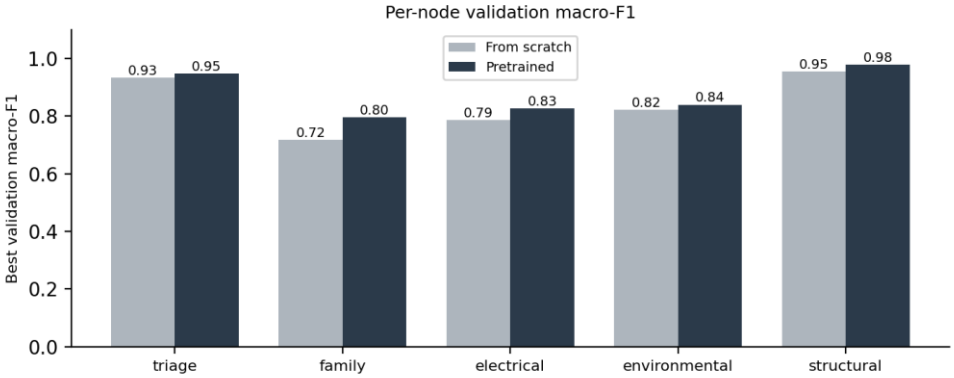


Fig. A.1. Best validation macro-F1 of each stage network, from scratch and pretrained.

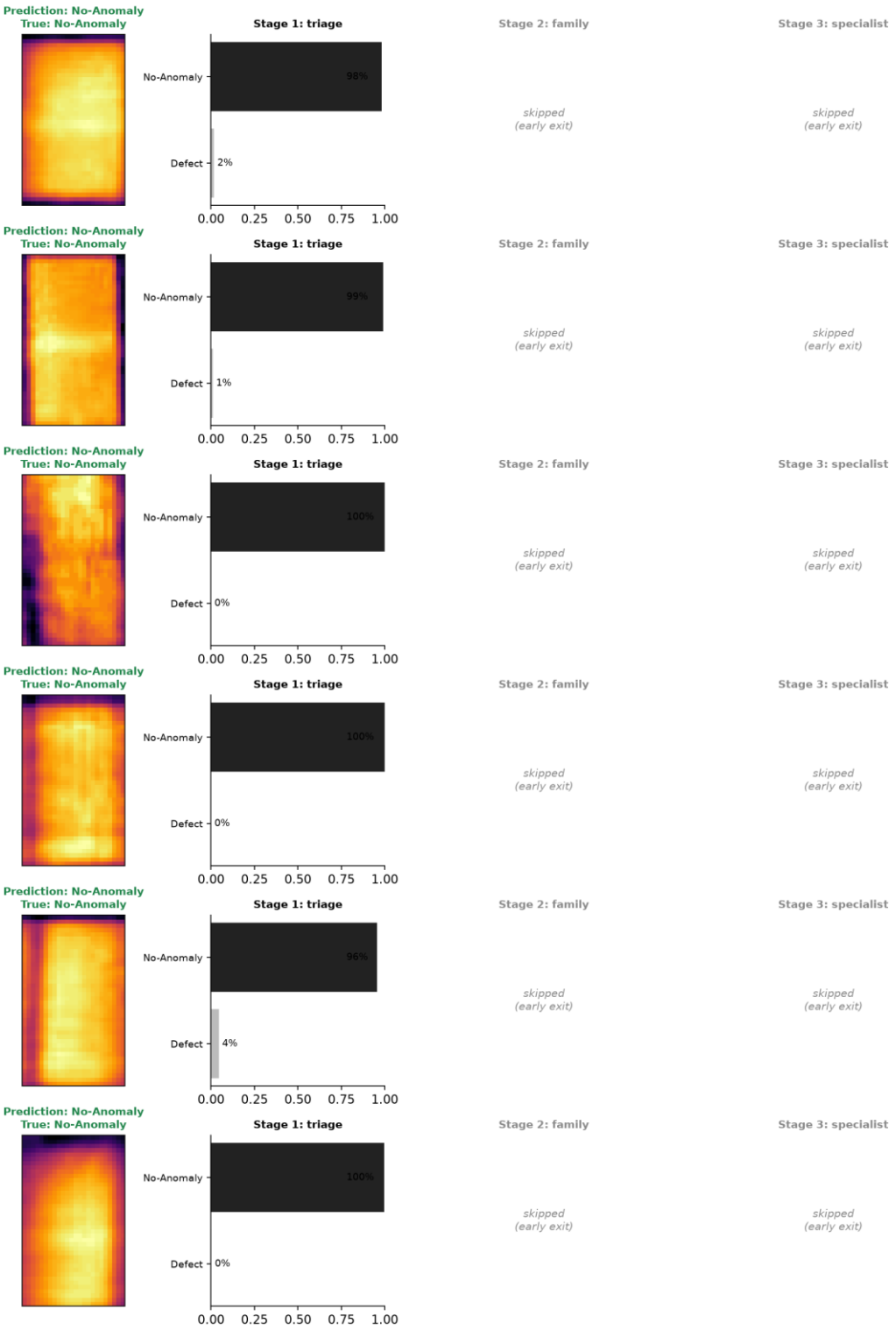


Fig. A.2. Output of the live demonstration for random test images (green: correct, red: wrong).